11/9/2024

PAGE :
DATE : / /

Devops Laboratory

→ Create folder → the create text → hide
git version
git init
git add
git commit -m "Initial commit"
git config --local/global user.name "_____"
git config --local/global user.email "_____"
git config --list
In new terminal, git remote add origin https://github.com/ of VScode
copy & past the 3 lines after creating the repository in the VS code terminal
git branch -M main
git push -u origin main

git branch
git checkout -b branch1
git branch
→ branch1
main
git switch main
git add
git commit -m "updated commit"
git switch branch1
git branch
git add
git commit -m "updated in branch1"
git switch main
git merge branch1

→ Accept both changes
→ Add some text

PAGE :
DATE : / /

git add .
git commit -m "updated in main branch"
git status.

Scanned with OKEN Scanner

18/9/24

PAGE :
DATE : / /

MAVEN Experiment

Step 1 : open command prompt & terminal
java -version
mvn -version

Step 2 : go to chrome open
Apache maven download
Then click the new version of Apache that is zip file.
from Binary zip archive.

Step 3 : extract files

Step 4 : copy address of bin

Step 5 : In "environment variables
change user path
Add new, paste address & save
Under system path
create JAVA_HOME & insert path.

Step 6 : check mvn -version

Step 7 : continue with the commands

24/9/2026

PAGE :
DATE : / /

Experiment-2

Automated Build & Dependency Management using Apache Maven :-

- javac -version
- mvn -version

creation of maven project template :

- mvn archetype:generate -DgroupId=com.example.app -DartifactId=demo-app -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

Template :-

demo app
- pom.xml → heart of MAVEN → dependencies, plugins
  - pom → project object model
- src
  - create different packages to prevent naming conflicts

- cd demo-app dir

- notepad pom.xml

Code for the pom.xml file :-

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/maven-v4_0_0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example.app</groupId>
    <artifactId>demo-app</artifactId>
    <packaging>jar</packaging>
    <version>1.0-SNAPSHOT</version>
```

Scanned with OKEN Scanner

```
<name> demo-app </name>
<url> http://maven.apache.org </url>
<dependencies>
<dependency>
    <groupId> ch.qos.logback </groupId>
    <artifactId> logback-classic </artifactId>
    <version> 1.1.1 </version>
</dependency>

<!-JUnit for testing-->
<dependency>
    <groupId> junit </groupId>
    <artifact> junit </artifactId>
<version> 4.13.2 </version>
<scope> test </scope>
</dependency>

<!--Logging-->
<dependency>
    <groupId> org.slf4j </groupId>
    <artifactId> slf4j-api </artifactId>
    <version> 2.0.9 </version>
</dependency>

<!-- Apache Commons->
  <dependency>
    <groupId> org.apache.commons </groupId>
    <artifactId> commons-lang3 </artifactId>
<version> 3.14.0 </version>
  </dependency>
</dependencies>
<build>
```

```
<plugins>
    <!-- Compiler Plugins -->
    <plugin>
        <groupId> org.apache.maven.plugins </groupId>
        <artifactId> maven-compiler-plugin </artifactId>
        <version> 3.11.0 </version>
        <configuration>
            <source> 17 </source>
            <target> 17 </target>
        </configuration>
    </plugin>
    <!-- Test Plugin -->
    <plugin>
        <groupId> org.apache.maven.plugins </groupId>
        <artifactId> maven-surefire-plugin </artifactId>
        <version> 3.1.2 </version>
    </plugin>
  </plugins>
</build>
</project>
```

- notepad src/main/java/com/example/app/App.java

Code of App.java :-

```
package com.example.app;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.apache.commons.lang3.StringUtils;

public class App {
    private static final Logger logger = LoggerFactory.getLogger(
                                        App.class);
```

Scanned with OKEN Scanner

```
public static void main(String[] args){
    String name = "CI/CD Pipeline";
    if(StringUtils.isNotBlank(name)){
        String message = greet(name);
        logger.info(message);
        System.out.println(message);
    } else
    {
        logger.error("Name is empty");
    }
}

public static String greet(String name){
    return "Hello, "+name+"! Welcome to Maven CI/CD
    Demo.";
}
}
```

- notepad src/test/java/com/example/app/AppTest.java

Code of AppTest.java:-

```
package com.example.app;
import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class AppTest{
    @Test
    public void testGreet(){
        String result = App.greet("Student");
        assertEquals("Hello, Student! Welcome to
        Maven CI/CD Demo.", result);
    }
}
```

Scanned with OKEN Scanner

PAGE :
DATE : / /

- mvn install clean
- mvn exec:java -Dexec.mainClass="com.example.App.App"

25/9/25

8/10/26

PAGE:
DATE: / /

Experiment :-

Application Containerization using Docker:-

Create a Dockerfile to package static web assets into an Apache Tomcat container, configuring the environment to serve HTML content & distributing the finalized image to a container registry.

1) Docker --version
2) Docker info
3) Docker run hello-world

type her

Docker Installation successful

4) Create folder tomcat-web-app
Create file → index.html using notepad index.html

Code of index.html

```
<!DOCTYPE html>
<html>
<head>
    <title> Docker Tomcat Application </title>
</head>
<body>
    <h1> Welcome to My Dockerized Web Application </h1>
    <p>
        This is a static web application running inside
        an Apache Tomcat Docker Container.
    </p>
    <h2> Application Containerization using Docker </h2>
    <p> Successfully deployed using docker & Apache Tomcat. </p>
</body>
</html>
```

Scanned with OKEN Scanner

5) Create file Dockerfile using notepad 'Dockerfile'

Code of Dockerfile :

```
FROM tomcat:10.1-jdk17          } pull from DockerHUB

COPY index.html /usr/local/tomcat/webapps/ROOT/index.html
                                  path of Container
EXPOSE 8080                     } Port no.

CMD ["catalina.sh", "run"]
```

(*) ren Dakefile.txt Dockerfile (Change the file type)

6) Docker build -t tomcat-web-app .
   (space + fullstop — indicates to build in current directory)

Docker images

7) Create / Instantiate the container.

Docker run -d -p 8080:8080 --name tomcat-web-container tomcat-web-app:latest
   (name of container)

Docker ps → shows the running jobs

8) Open localhost:8080.
Content within the index.html will be displayed

PAGE :
DATE : / /

Command to push to Docker Hub,

Docker push

Docker tag tomcat-web-app:latest prakriaye/tomcat-web-app:latest

Docker push (copy image name)
(get by using command Docker images

Befor push use
Docker login
& authorize, then use push.

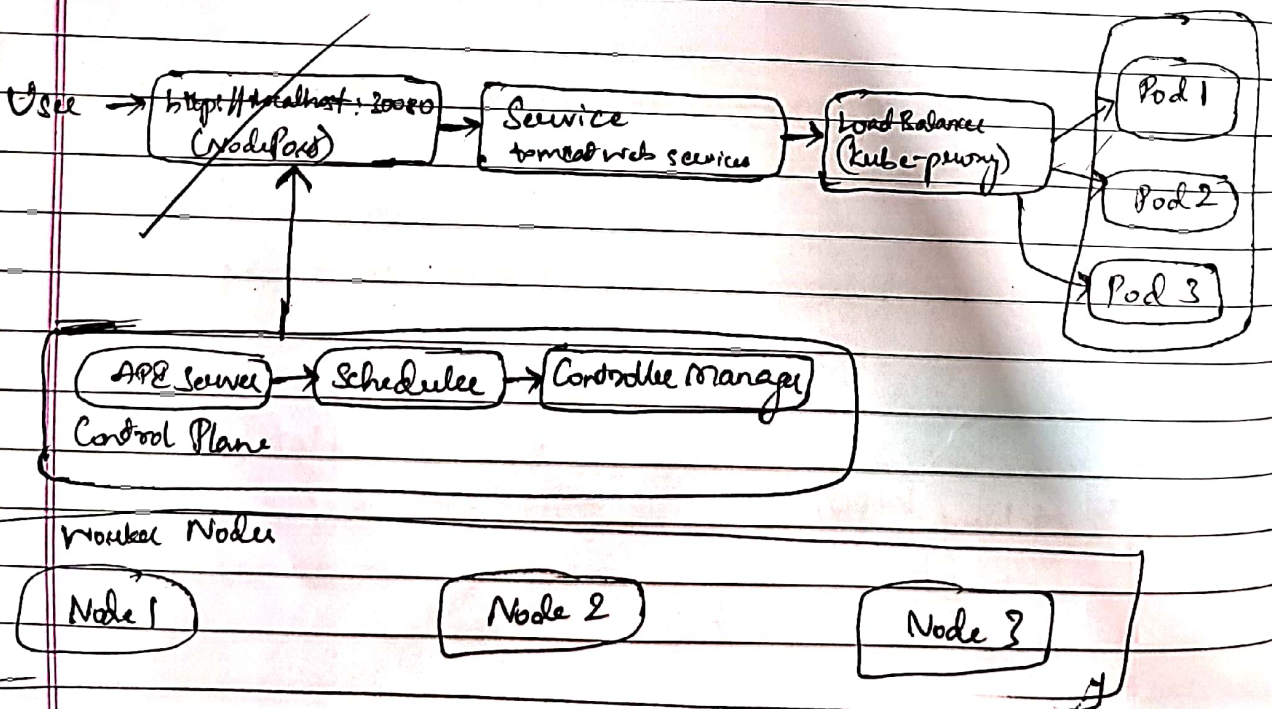
16/10/24

16/10/2026

PAGE :
DATE :

Experiment - 5

- Install Docker Desktop. (select 2nd option) → Pull on local system
  - ↳ WSL
- Create K8S
  - ↳ Kind → 4 nodes
1) → kubectl get nodes
   - ↳ list all the nodes in the K8S cluster
2) → notepad deployment.yaml
3) → kubectl apply -f deployment.yaml
   - ↳ kubectl get pods
   - → 3 pods
4) → notepad service.yaml
5) → kubectl apply -f service.yaml
   - ↳ kubectl get service
   - → K8S
   - → tomcat-service
   - ↳ open another terminal
     - ↳ invoke.



Scanned with OKEN Scanner

16/10/2026

PAGE :
DATE : / /

6). Kubectl port-forward service/tomcat-web-service 8081:8080

=> Open another terminal → Run on powershell

7). Invoke-webRequest http://localhost:8081 -UseBasicParsing → load balancing

Open browser & check the URL.

- Kubectl Scale deployment tomcat-web-deployment --replicas=5 → scaling
- Kubectl get pods -w
- Kubectl delete pod tomcat-web-deployment-7585d8745-fsdvb

Kubectl get endpoints slices

16/10/26.

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: tomcat-web-deployment

spec:
  replicas: 3
  selector:
    matchLabels:
      app: tomcat-web
  template:
  metadata:
```

Scanned with OKEN Scanner

```
    labels:
      app: tomcat-web
  spec:
    containers:
    - name: tomcat-web
      image: tomcat-web-app:latest
      imagePullPolicy: IfNotPresent
      ports:
        - containerPort: 8080
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: tomcat-web-service
spec:
  type: NodePort
  selector:
    app: tomcat-web
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
      nodePort: 30080
```

30100

PAGE :
DATE : / / /

Experiment 4

HCL → HashiCorp Language (Configuration)

Requirements

- Download Terraform for Windows
- Unzip
- Copy unzipped folder to Program files in C drive.
- Copy the path of that (copied) folder
- Add that path to PATH in system variables
  (double click on it then add path)

Open new Command Prompt.

terraform --version

- Create Folder in D drive → DaC Terraform
  (Infrastructure as Code)
- Create file main.tf

```
terraform {
  required_providers {
    docker = {
      source = "kreuzwerker/docker"
      version = "~> 4.2"
    }
  }
}

provider "docker" {}
resource "docker_image" "nginx" {
  name = "nginx:latest"
  keep_locally = true
}
```

(~ tilde)

Scanned with OKEN Scanner

PAGE :
DATE : / /

```
resource "docker_container" "web" {
  name = "terraform-web-server"
  image = docker_image.nginx.image_id

  ports {
    internal = 80
    external = 8081
  }
}

output "container_name" {
  value = docker_container.web.name
}
output "web_url" {
  value = "http://localhost:8081"
}
```

(#) convert main file from .txt to .tf in copy of that folder.
ren main.tf.txt main.tf

Open terminal inside that folder :

→ terraform init (Successful Initialization)

→ terraform validate (Success! Config. is valid)

→ terraform fmt (Formatting the file/terraform no output message)

→ terraform plan (Output ++++) [One warning will pop up → ignore]

Scanned with OKEN Scanner

→ terraform apply [Enter value: yes]

(Your container will reflect on Docker)

(*) Keep Docker Engine Running while building this project.

→ Check in docker desktop

→ In chrome localhost:8081

→ In terminal;

dockerps

shows container id, image, command etc.

→ terraform state list

2 lines

→ terraform show

make changes in the main.tf

→ Change the port from 8081 - 8085 in notepad

In same terminal

terraform plan

o/p:- port will be changed.

→ terraform apply

o/p: Enter a value: yes

1 added, 0 changed, 1 destroyed

→ terraform destroy

30/10/24 o/p: Enter a value: yes